

# Unit 3 Analytical Answers

---

## Unit 3 Analytical Questions

192521079

## 1. MD5 — Analytical Problems

---

### Problem 1 — MD5 Padding and Block Calculation

**Question:** For the message “HELLO”, convert to ASCII hex, find the length in bits, apply MD5 padding, append the 64-bit length field, and determine the number of 512-bit blocks.

**Answer:**

1. ASCII hex of “HELLO”: 48 45 4C 4C 4F
2. Length: 5 bytes = **40 bits**
3. MD5 padding always appends a single 1 bit followed by enough 0 bits so the message length  $\equiv 448 \pmod{512}$ . 40 bits + 1 bit + 407 zero bits = 448 bits
4. Append the 64-bit field encoding the original length (40, in little-endian for MD5):  $448 + 64 = 512$  bits
5. **Total blocks =  $512 / 512 = 1$  block**

The padded message therefore fits exactly into a single 512-bit MD5 block.

### Problem 2 — MD5 Compression Analysis

**Question:** Given  $A=67452301$ ,  $B=EFCDA89$ ,  $C=98BADCFE$ ,  $D=10325476$ , compute  $F(B,C,D) = (B \text{ AND } C) \text{ OR } ((\text{NOT } B) \text{ AND } D)$ .

**Answer:**

Working nibble-by-nibble in binary:

- **B AND C:**  $EFCDA89 \text{ AND } 98BADCFE = 88888888$
- **NOT B:** complement of  $EFCDA89 = 10325476$  (note this happens to equal D exactly, since these constants are complementary pairs)
- **(NOT B) AND D:**  $10325476 \text{ AND } 10325476 = 10325476$
- **$F(B,C,D) = 88888888 \text{ OR } 10325476 = 98BADCFE$**

**$F(B,C,D) = 0x98BADCFE$**

**Role in the round:** F is one of four nonlinear “choice” functions used in MD5’s 64-step compression. For each bit position, F acts as a bitwise selector: wherever B has a 1, the output bit is taken from C; wherever B has a 0, the output bit is taken from D. This makes F a bit-conditional multiplexer. The result is added (mod  $2^{32}$ ) to A, the message word, a round constant, then rotated and added to B to produce the new state.

**Why nonlinear mixing matters:** If MD5 only used linear operations (XOR, addition), the transformation could be attacked using linear algebra over  $GF(2)$ , since XOR/addition-only systems are much easier to invert or predict. AND/OR/NOT-based selection functions like F break this linearity, so small changes in input bits do not propagate predictably through the round — this diffusion and confusion is essential to resist cryptanalysis (although, as Problem 3 shows, MD5’s overall design is still broken today).

### Problem 3 — MD5 Security Analysis

**Question:** Two different messages M1 and M2 produce the same MD5 digest 5d41402abc4b2a76b9719d911017c59 . What does this show?

**Answer:**

1. **Security property violated: Collision resistance.** A cryptographic hash function should make it computationally infeasible to find any two distinct inputs that map to the same output. Here two different messages collide, so collision resistance has failed.
2. **Suitable for digital signatures?** No. Digital signatures rely on the impossibility of finding a second message with the same hash as a signed message. If collisions are practical, an attacker can substitute a malicious document that hashes to the same value as a legitimately signed one, so **MD5 must not be used for signatures**.
3. **Comparison of the three properties:**

| Property                   | Definition                                               | Status for MD5                               |
|----------------------------|----------------------------------------------------------|----------------------------------------------|
| Collision resistance       | Hard to find any $x_1 \neq x_2$ with $H(x_1)=H(x_2)$     | <b>Broken</b> (seconds, on a laptop)         |
| Preimage resistance        | Given $h$ , hard to find $x$ with $H(x)=h$               | Weakened but not fully broken practically    |
| Second-preimage resistance | Given $x_1$ , hard to find $x_2 \neq x_1$ with same hash | Weakened, more costly than collision attacks |

4. **Why MD5 is insecure today:** Its 128-bit digest gives only  $2^{64}$  ideal collision resistance, and structural weaknesses in its compression function (found by Wang et al., 2004) allow chosen-prefix collisions to be generated in seconds. Because collisions are cheap and adversarially constructible (not just randomly probable), MD5 cannot be trusted anywhere collision resistance matters — digital signatures, certificate signing, or software integrity.

## 2. SHA-1 — Analytical Problems

### Problem 1 — SHA-1 Message Padding

**Question:** For “ABC”, convert to binary/hex, find length, apply SHA-1 padding, determine padded size, and explain the final 64 bits.

**Answer:**

1. “ABC” in hex: 41 42 43 ; in binary: 01000001 01000010 01000011
2. Length: 3 bytes = **24 bits**
3. SHA-1 padding: append a single 1 bit, then 0 bits until length  $\equiv 448 \pmod{512}$ :  $24 + 1 + 423$  zero bits = 448 bits
4. Append the 64-bit big-endian length field (value = 24):  $448 + 64 =$  **512 bits total**
5. **Why the final 64 bits store the length:** This is the Merkle-Damgård strengthening technique. Encoding the original message length at a fixed, known position at the end of the padded message prevents trivial padding/length-extension ambiguities and defeats simple collision attacks that exploit variable-length padding (e.g., an attacker appending garbage that still parses as valid padding). It fixes the total input length unambiguously for every message that hashes to the same padded size.

### Problem 2 — SHA-1 Word Expansion

**Question:** Given  $W_0=12345678$ ,  $W_2=ABCDEF01$ ,  $W_8=11111111$ ,  $W_{13}=22222222$ , compute  $W_t = \text{ROTL1}(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16})$  for the word that uses exactly these four inputs.

**Answer:**

These four indices (0, 2, 8, 13) match the formula for  $t = 16$ , since  $16-3=13$ ,  $16-8=8$ ,  $16-14=2$ ,  $16-16=0$ .

**Step 1 — XOR all four words** (nibble-by-nibble binary XOR):

```
W13  ⊕  W8   ⊕  W2   ⊕  W0
22222222 ⊕ 11111111 = 33333333
```

```
33333333 ⊕ ABCDEF01 = 99FE1D32 (intermediate check)
99FE1D32 ⊕ 12345678 = 8ACA4B4A ...
```

Carrying out the full bitwise XOR of all four 32-bit words precisely gives:

**W13 ⊕ W8 ⊕ W2 ⊕ W0 = 0x8ACA8A4A**

**Step 2 — Rotate left by 1 bit (ROTL1):** shift all bits left by one position; the vacated bit on the right is filled by the bit that fell off the left (the MSB).

8ACA8A4A in binary = 1000 1010 1100 1010 1000 1010 0100 1010 Rotating left by 1 (MSB 1 wraps to the LSB position): → 0001 0101 1001 0101 0001 0100 1001 0101

**W16 = ROTL1(0x8ACA8A4A) = 0x15951495**

This expanded word W16 then becomes an input to round 16 of SHA-1's 80-round compression function, illustrating how SHA-1 stretches the initial 16 message words into 80 words using XOR and rotation, giving each round fresh, message-dependent material.

### Problem 3 — SHA-1 Security Analysis

**Question:** An attacker finds a practical collision against SHA-1 used for integrity-checking downloaded software.

**Answer:**

- Effect on the system:** An attacker can craft two different files — a legitimate one and a malicious one — that hash to the identical SHA-1 digest. If the system trusts “matching hash ⇒ same/untampered file,” it will accept the malicious file as if it were the verified original, defeating the integrity check entirely (this is essentially what the real-world “SHAttered” attack demonstrated against SHA-1 in 2017).
- Property that failed:** Collision resistance.
- Why SHA-256 is preferred:** SHA-256 has a larger 256-bit digest ( $2^{128}$  ideal collision resistance vs. SHA-1's  $2^{80}$ ), a more complex/refined compression function, and — critically — no practical collision attack has been demonstrated against it. It belongs to the SHA-2 family, which remains considered secure.
- Would widening SHA-1's output from 160 to 320 bits fix it?** No. Simply storing or concatenating more output bits does not repair the underlying weakness in SHA-1's compression function/message schedule that enables differential collision attacks. The vulnerability is structural, not a matter of digest length; you cannot patch a broken algorithm by making its output longer — you must replace the algorithm itself (e.g., migrate to SHA-256/SHA-3).

## 3. SHA-224 — Analytical Problems

### Problem 1 — Output Size Analysis

| Algorithm | Digest size | Hex characters | Ideal collision resistance | Ideal preimage resistance |
|-----------|-------------|----------------|----------------------------|---------------------------|
| SHA-224   | 224 bits    | 56             | $\approx 2^{112}$          | $\approx 2^{224}$         |
| SHA-256   | 256 bits    | 64             | $\approx 2^{128}$          | $\approx 2^{256}$         |

- Digest size difference:  $256 - 224 = 32$  bits
- Hex character difference:  $64 - 56 = 8$  characters
- Collision resistance follows the birthday bound (digest/2), preimage resistance follows the full digest size.

### Problem 2 — Padding Analysis (1000-bit message)

1. **Blocks before padding:**  $1000 \text{ bits} < 512 \times 2$ , so it needs at least  $\lceil 1000/512 \rceil = 2$  blocks worth of space, i.e. 1 full block is not enough.
2. **Padding bits required:** SHA-224 (like SHA-256) pads to 448 (mod 512).  $1000 \bmod 512 = 488$ . Since  $488 > 448$ , padding must roll into the next 512-bit block: pad up to  $448 + 512 = 960$ , so padding bits =  $960 - 1000 \pmod{512}$  arithmetic  $\rightarrow$  concretely: append 1 bit + zeros until reaching 960 bits total (before the length field):  $960 - 1000 \dots$  let's compute directly: after the message (1000 bits) add a 1 bit  $\rightarrow 1001$  bits, then zeros until the total  $\equiv 448 \pmod{512}$ . The smallest multiple satisfying this above 1001 is  $960 + 512 = 1472$ ? Re-deriving cleanly: we need total  $\equiv 448 \pmod{512}$  and total  $\geq 1001$ .  $448 \bmod 512$  itself (448) is less than 1001, so the next valid target is  $448 + 512 = 960$  (still  $< 1001$ ), next is  $960 + 512 = 1472$ . So padding zeros =  $1472 - 1001 = 471$  zero bits (plus the 1 initial bit = 472 padding bits total).
3. **Final blocks after adding the 64-bit length field:**  $1472 + 64 = 1536$  bits =  $1536/512 = 3$  blocks.
4. **Why an extra block may be needed:** Because the original message length (1000 bits) already exceeds the 448-bit threshold within its first 512-bit block, there isn't room left in that block for the mandatory 1 bit, zero padding, and 64-bit length field. The padding must spill into a second full block, which is why messages just under two block-lengths sometimes need a third block purely for padding/length overhead — this is identical mechanics to SHA-256 since both share the same padding scheme (only initial hash values and final truncation differ).

### Problem 3 — SHA-224 vs SHA-256

**Requirement:** at least “ideal” collision resistance with the shortest possible digest.

**Answer:** SHA-224 satisfies a lower resistance level ( $2^{112}$ ) than SHA-256 ( $2^{128}$ ), but  $2^{112}$  is still well beyond any currently feasible brute-force or birthday attack, and it remains part of the actively-maintained SHA-2 family (not deprecated like MD5/SHA-1).

- **Digest size:** SHA-224 is 32 bits shorter — smaller storage footprint, useful for constrained systems or protocols that cap field sizes.
- **Collision resistance:** SHA-256 offers a larger security margin ( $2^{128}$  vs  $2^{112}$ ), but  $2^{112}$  already vastly exceeds practical attack budgets.
- **Storage:** SHA-224 saves  $\sim 12.5\%$  of storage/bandwidth per hash, which matters at scale (e.g., certificate chains, blockchains with millions of hashes).
- **Security margin:** If the organization’s requirement is strictly “at least ideal collision resistance” with no specific numeric floor beyond that, **SHA-224 is the more appropriate, storage-efficient choice**. If future-proofing against advances in cryptanalysis is a priority, SHA-256 is safer due to its larger margin.

## 4. SHA-256 — Analytical Problems

### Problem 1 — SHA-256 Padding

**Question:** “HELLO WORLD” — find length, pad, append 64-bit length, count blocks.

**Answer:**

1. “HELLO WORLD” = 11 characters = **88 bits**
2. Padding: append 1 bit + zeros until  $\equiv 448 \pmod{512}$ :  $88 + 1 + 359 = 448$  bits
3. Append 64-bit length field (value 88):  $448 + 64 = 512$  bits
4. **Number of 512-bit blocks = 1**

### Problem 2 — SHA-256 $\Sigma_0$ Function

**Question:** For  $x = 6A09E667$ , compute  $\Sigma_0(x) = \text{ROTR2}(x) \oplus \text{ROTR13}(x) \oplus \text{ROTR22}(x)$ .

**Answer (computed and verified bit-by-bit):**

| Operation | Result |
|-----------|--------|
|-----------|--------|

|                                                    |                   |
|----------------------------------------------------|-------------------|
| ROTR2(x)                                           | 0xDA827999        |
| ROTR13(x)                                          | 0x333B504F        |
| ROTR22(x)                                          | 0x27999DA8        |
| <b><math>\Sigma_0(x)</math> = XOR of all three</b> | <b>0xCE20B47E</b> |

( $x = 0x6A09E667$  is in fact SHA-256's initial hash constant  $H_0$ , and  $CE20B47E$  matches the standard reference-implementation value for  $\Sigma_0$  of this input — a useful sanity check.)

**Why rotation + XOR instead of simple shifting?** A plain logical shift discards bits off one end and loses information permanently — repeated shifting would eventually zero out the whole word, destroying entropy. A *rotation* is bijective (reversible) and preserves every bit, just repositioning them, so no information is lost even after many rounds. Combining three different rotation amounts with XOR ensures every output bit depends on multiple, well-separated input bits (good diffusion), while remaining efficient to compute in hardware/software (rotation is a single cheap CPU instruction). This nonlinear mixing of position is what gives SHA-256's compression function its avalanche effect and resistance to differential cryptanalysis.

### Problem 3 — SHA-256 vs MD5 (banking application)

| Feature                 | MD5                                        | SHA-256                                         |
|-------------------------|--------------------------------------------|-------------------------------------------------|
| Digest length           | 128 bits                                   | 256 bits                                        |
| Word size               | 32 bits                                    | 32 bits                                         |
| Number of rounds        | 64                                         | 64 (but larger, more complex round function)    |
| Collision resistance    | <b>Broken</b> (practical collisions exist) | No known practical attack; $\sim 2^{128}$ ideal |
| Current security status | Deprecated / insecure                      | Actively recommended (SHA-2 family)             |

**Recommendation:** The bank should migrate away from MD5 entirely to **SHA-256** (or better, a purpose-built password hashing function such as bcrypt/scrypt/Argon2 if the use case is specifically password storage, since general-purpose fast hashes like SHA-256 are vulnerable to GPU-accelerated brute force on passwords). For generic file/data integrity checks, SHA-256 is the appropriate modern replacement for MD5.

## 5. SHA-384 — Analytical Problems

### Problem 1 — SHA-384 Output Calculation

**Answer:**

- 96 hex characters  $\times$  4 bits/char = **384 bits**
- $384 / 8 =$  **48 bytes**
- Compared to SHA-256's 256 bits (32 bytes), SHA-384 is 128 bits (16 bytes) longer.
- Advantage of the longer digest:** A larger digest raises the theoretical collision-resistance bound ( $2^{192}$  vs SHA-256's  $2^{128}$ ) and preimage-resistance bound, giving a larger long-term security margin against future cryptanalytic advances or quantum attacks (Grover's algorithm halves effective preimage security, so a bigger digest keeps more headroom).

### Problem 2 — SHA-384 Block Analysis (1000-byte message)

- Message size: 1000 bytes  $\times$  8 = **8000 bits**
- SHA-384 uses 1024-bit blocks. 8000 bits needs padding to a multiple of 1024 (with room for a 128-bit length field, padded to  $\equiv 896 \pmod{1024}$ ).  $8000 \pmod{1024} = 704$ , which is less than 896, so it fits with padding into:  $\lceil (8000+1+128)/1024 \rceil \rightarrow$  concretely the message fits into **8 blocks** (since 8000 bits alone almost fills 7.8 blocks; the padding plus 128-bit length field pushes the total to  $8 \times 1024 = 8192$  bits).
- Why 1024-bit blocks instead of 512:** SHA-384 (like SHA-512) operates on 64-bit words internally (8

words  $\times$  64 bits = 512 bits of state, processed with a 1024-bit message block — double the block size of SHA-256’s 32-bit-word design). Larger blocks and 64-bit word arithmetic let it process more data per round and take advantage of native 64-bit CPU registers, improving throughput on modern processors.

4. **Comparison with SHA-256:** SHA-256 uses 32-bit words, 512-bit blocks, 64 rounds; SHA-384 uses 64-bit words, 1024-bit blocks, 80 rounds, and a 128-bit length field (vs SHA-256’s 64-bit length field) to accommodate far larger messages.

### Problem 3 — SHA-384 Security Analysis

**Statement to evaluate:** “SHA-384 is always twice as secure as SHA-256 because  $384 > 256$ .”

**Answer:** This is an **oversimplification and not technically accurate**.

- **Collision resistance:** SHA-384 gives  $\sim 2^{192}$  vs SHA-256’s  $\sim 2^{128}$  — a large increase, but not literally “double” in a linear sense; security strength scales with the *exponent*, not the digest length itself. Going from 128-bit to 192-bit security is an enormous ( $2^{64}\times$ ) increase in work factor, far more than “double.”
- **Preimage resistance:** Similarly scales exponentially with digest size, not linearly.
- **Digest size:** 384 bits is  $1.5\times$  the *length* of 256 bits, not  $2\times$ , so even the raw length comparison in the statement is arithmetically off.
- **Computational cost:** SHA-384 is more expensive to compute per byte (larger internal state, 64-bit arithmetic), so the extra margin comes at a real performance cost — “more secure” isn’t free.
- **Conclusion:** Bit-length comparisons don’t map linearly to “security multiples.” The correct framing is that SHA-384’s security level is exponentially larger, not simply “ $2\times$ ,” and whether that extra margin is *necessary* depends on the threat model — for most applications SHA-256’s 128-bit security is already considered computationally unbreakable for the foreseeable future.

## 6. SHA-512 — Analytical Problems

### Problem 1 — SHA-512 Padding (L = 900 bits)

1. **Blocks before padding:** 900 bits  $<$  1024, so it currently occupies part of **1 block** (not yet full).
2. **Padding requirement:** SHA-512 pads to  $\equiv 896 \pmod{1024}$ . Since  $900 > 896$ , padding must spill into a second block: append a **1** bit + zeros until reaching  $896 + 1024 = 1920$  bits (the next valid target above 901).
3. **Length-field size:** SHA-512 reserves a **128-bit** length field (large enough to encode messages up to  $2^{128}$  bits — far larger than SHA-256’s 64-bit field), to match its bigger block size and support extremely large inputs.
4. **Final number of blocks:**  $1920 + 128 = 2048$  bits  $\rightarrow 2048 / 1024 =$  **2 blocks**.

### Problem 2 — SHA-512 Word Operation

**Question:**  $x = 6A09E667F3BCC908$ , compute  $\Sigma_0(x) = \text{ROTR1}(x) \oplus \text{ROTR8}(x) \oplus \text{SHR7}(x)$ .

**Answer (64-bit arithmetic, computed and verified):**

| Operation     | Result             |
|---------------|--------------------|
| ROTR1(x)      | 0x3504F333F9DE6484 |
| ROTR8(x)      | 0x086A09E667F3BCC9 |
| SHR7(x)       | 0x00D413CCCFE77992 |
| $\Sigma_0(x)$ | 0x3DBAE91951CAA1DF |

( $x = 0x6A09E667F3BCC908$  is SHA-512’s initial hash constant  $H_0$ .)

**Effect of 64-bit words vs SHA-256’s 32-bit words:** Doubling the word size doubles the internal state per register and the per-operation data throughput, which is a natural fit for modern 64-bit processors (fewer

instructions needed to move the same amount of data). It also widens the practical security margin (larger word-level mixing space) and lets SHA-512 use a bigger 128-bit length field. The trade-off is that on 32-bit or embedded hardware without native 64-bit ALU support, SHA-512 can actually run *slower* than SHA-256, since each 64-bit operation must be emulated with two 32-bit operations.

Problem 3 — SHA-512 vs SHA-256 (large-data server)

| Feature                    | SHA-256                              | SHA-512                                                       |
|----------------------------|--------------------------------------|---------------------------------------------------------------|
| Word size                  | 32-bit                               | 64-bit                                                        |
| Block size                 | 512 bits                             | 1024 bits                                                     |
| Digest size                | 256 bits                             | 512 bits                                                      |
| Rounds                     | 64                                   | 80                                                            |
| Performance on 64-bit CPUs | Good, but underuses 64-bit registers | <b>Better</b> — native 64-bit ops process more data per cycle |
| Security strength          | $\sim 2^{128}$                       | $\sim 2^{256}$                                                |

**Recommendation:** For a modern 64-bit server processing very large volumes of data, **SHA-512** is generally the better choice: its 64-bit word operations map directly onto native CPU registers, often making it *faster* than SHA-256 on such hardware despite doing more work per block, while also providing a larger security margin. (If interoperability or a specific 256-bit output is required, SHA-512/256 offers SHA-512’s performance benefits with a 256-bit digest — see Section 7.)

7. SHA-512/224 and SHA-512/256 — Analytical Problems

Problem 1 — Digest Comparison

| Algorithm   | Digest size | Internal processing                              |
|-------------|-------------|--------------------------------------------------|
| SHA-224     | 224 bits    | 32-bit word, SHA-256-style                       |
| SHA-256     | 256 bits    | 32-bit word                                      |
| SHA-512/224 | 224 bits    | 512-bit internal (64-bit word), truncated output |
| SHA-512/256 | 256 bits    | 512-bit internal (64-bit word), truncated output |

**Why SHA-512/224 can output 224 bits while using SHA-512 internally:** It runs the *full* SHA-512 compression algorithm (64-bit words, 1024-bit blocks, 80 rounds) but with a distinct set of initial hash values (IVs), and then simply **truncates** the final 512-bit output down to the first 224 bits before returning it. Truncation of a well-mixed final digest doesn’t weaken the algorithm structurally as long as unique IVs are used (this also protects against length-extension-style cross-protocol attacks, since the truncated output can’t easily be extended back into the full internal state).

Problem 2 — 224-bit hash on 64-bit processors

| Aspect             | SHA-224  | SHA-512/224 |
|--------------------|----------|-------------|
| Internal word size | 32-bit   | 64-bit      |
| Block size         | 512 bits | 1024 bits   |
| Digest size        | 224 bits | 224 bits    |
|                    |          |             |



|                        |                                     |                                                                          |
|------------------------|-------------------------------------|--------------------------------------------------------------------------|
| Security level         | $\sim 2^{112}$ collision resistance | $\sim 2^{112}$ collision resistance (same target, larger internal state) |
| Processing environment | Efficient on 32-bit systems         | <b>Efficient on 64-bit systems</b> (native word ops)                     |

**More suitable:** For a company running mainly on 64-bit processors, **SHA-512/224** is the more appropriate choice — it delivers the same 224-bit output and comparable security target as SHA-224, but its 64-bit-word internal operations run faster on 64-bit hardware.

**Problem 3 — Is SHA-512/256 unnecessary given SHA-256 already exists?**

**Answer:** The developer’s claim is **not correct**. Even though SHA-512/256 and SHA-256 both produce 256-bit digests, they are built on very different internal engines:

- SHA-512/256 uses SHA-512’s 64-bit-word, 1024-bit-block structure, so on 64-bit server/desktop hardware it can be noticeably **faster** than SHA-256 while still producing a compact, storage-friendly 256-bit digest.
- It also inherits SHA-512’s larger internal state (512-bit chaining value truncated to 256 bits), which some argue gives extra resistance against certain attacks compared to a hash whose internal state is the same size as its output.
- So SHA-512/256 is useful specifically as a “best of both worlds” option: SHA-256-sized output with SHA-512-speed internals on 64-bit systems — not redundant, but a legitimate performance-oriented alternative.

**8. SHA Version Comparison — Analytical Problems**

**Problem 1 — Complete Comparison Table**

| Feature         | MD5                        | SHA-1                      | SHA-224        | SHA-256                     | SHA-384                     | SHA-512                     |
|-----------------|----------------------------|----------------------------|----------------|-----------------------------|-----------------------------|-----------------------------|
| Digest size     | 128 bits                   | 160 bits                   | 224 bits       | 256 bits                    | 384 bits                    | 512 bits                    |
| Word size       | 32 bits                    | 32 bits                    | 32 bits        | 32 bits                     | 64 bits                     | 64 bits                     |
| Block size      | 512 bits                   | 512 bits                   | 512 bits       | 512 bits                    | 1024 bits                   | 1024 bits                   |
| Rounds          | 64                         | 80                         | 64             | 64                          | 80                          | 80                          |
| Security status | <b>Broken</b> — do not use | <b>Broken</b> — deprecated | Secure (SHA-2) | Secure (SHA-2), widely used | Secure (SHA-2), high margin | Secure (SHA-2), high margin |

**Problem 2 — Algorithm Selection Scenario**

| Use case                                  | Recommended approach                                                                           | Why                                                                                                                                                                           |
|-------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A. File integrity</b>                  | SHA-256 (or SHA-512 on 64-bit systems)                                                         | Strong collision resistance, no known practical attacks, wide tool support                                                                                                    |
| <b>B. Digital signatures</b>              | SHA-256 or SHA-384/512, paired with RSA/ECDSA                                                  | Signature security depends entirely on collision resistance of the underlying hash                                                                                            |
| <b>C. Password storage</b>                | A dedicated password-hashing function: <b>bcrypt, scrypt, or Argon2</b> (not a raw SHA-2 hash) | General-purpose hashes like SHA-256 are <i>too fast</i> , making brute-force/GPU cracking of passwords cheap; slow, memory-hard KDFs are designed specifically to resist this |
| <b>D. Blockchain-style data integrity</b> | SHA-256 (as used by Bitcoin) or SHA-3/Keccak variants                                          | Proven collision resistance and broad ecosystem adoption for chained-hash integrity                                                                                           |



**Why MD5 and SHA-1 should be avoided for new systems:** Both have publicly demonstrated, practical collision attacks (MD5 since 2004, SHA-1 since 2017's "SHAttered" attack), meaning an adversary can deliberately construct two different inputs sharing the same hash. This breaks the core security guarantee that integrity checks and signatures rely on, so continuing to use them in new designs introduces an avoidable, well-documented vulnerability.

### Problem 3 — Hash Security Requirement Evaluation

**Requirements:** Collision resistance  $\geq 2^{120}$ , Preimage resistance  $\geq 2^{250}$ , Digest  $\leq 256$  bits.

| Algorithm | Collision resistance                   | Preimage resistance | Digest $\leq 256$ bits? | Meets all requirements?                                                                           |
|-----------|----------------------------------------|---------------------|-------------------------|---------------------------------------------------------------------------------------------------|
| MD5       | $\sim 2^{64}$ (and practically broken) | $\sim 2^{128}$      | Yes (128 bits)          | <b>Reject</b> — fails collision requirement and is practically broken                             |
| SHA-1     | $\sim 2^{80}$ (practically broken)     | $\sim 2^{160}$      | Yes (160 bits)          | <b>Reject</b> — fails collision requirement ( $< 2^{120}$ ) and is practically broken             |
| SHA-224   | $\sim 2^{112}$                         | $\sim 2^{224}$      | Yes (224 bits)          | <b>Reject</b> — collision resistance ( $2^{112}$ ) falls just short of the $2^{120}$ requirement  |
| SHA-256   | $\sim 2^{128}$                         | $\sim 2^{256}$      | Yes (256 bits)          | <b>Accept</b> — meets both thresholds and fits within the digest-size cap                         |
| SHA-384   | $\sim 2^{192}$                         | $\sim 2^{384}$      | <b>No</b> (384 bits)    | <b>Reject</b> — exceeds the 256-bit digest-size limit, despite exceeding both security thresholds |
| SHA-512   | $\sim 2^{256}$                         | $\sim 2^{512}$      | <b>No</b> (512 bits)    | <b>Reject</b> — exceeds the 256-bit digest-size limit, despite exceeding both security thresholds |

**Conclusion:** Only **SHA-256** satisfies all three constraints simultaneously — strong enough collision and preimage resistance, while staying within the 256-bit digest-size ceiling. SHA-384 and SHA-512 are cryptographically stronger but disqualified purely by the digest-size cap; MD5, SHA-1, and SHA-224 are disqualified on security grounds.